

Java

- James goseling.

⇒ In object oriented programming the code we write is completely based upon object. An object is an incidence of class in other words an object is a real world entity.

Features of oops:

There are mainly 4 Features are there in oops.

1. Data encapsulation.
2. Data Abstraction.
3. Polymorphism
4. Inheritance.

* Data encapsulation:-

Binding up of code and data and putting at one place is known as data encapsulation.

Ex:- Class.

* Data Abstraction:-

Hiding up of the data at the same time showing only necessary data is known as Data Abstraction.

* Polymorphism:

→ Ability to exist in many forms.

there are 2 types of polymorphism are there.

1. Compile-time polymorphism.

Ex:- method overloading.

2. Runtime polymorphism

Ex:- method overriding.

* Inheritance:-

Deriving a child class from the parent class is known as inheritance.

(or)

Inheriting the properties of parent to the child is known as inheritance.

Programs

1. write a java program to print the message hello.

```
=> class First
{
    -----
    Public Static void main (String args[])
    {
        System.out.println ("hello");
    }
}
```

*. write a java program to print the sum of 2 numbers.

```
class First
```

```
{
```

```
    public static void main(String args[])
```

```
    {
```

```
        int a=20, b=20, c;
```

```
        c=a+b;
```

```
        System.out.println(c);
```

```
    }
```

```
}
```

*. write a java program to check wheather it is even or odd.

```
Public class First
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        int a=20;
```

```
        if (a%2==0)
```

```
            System.out.println("even");
```

```
        else
```

```
            System.out.println("odd");
```

```
    }
```

*. write a java program to store your name and print.

```
Public class main
```

```
{
```

```
    public static void main(String args[])
```

```
    {
```

```
String a = "India";
```

```
System.out.println(a);
```

```
}
```

```
}
```

```
* public class First
```

```
{
```

```
    public static void main (String args[])
```

```
{
```

```
    float a = 12.5f;    → double a = 12.5;
```

```
    System.out.println(a);
```

```
}
```

```
}
```

```
* public class First
```

```
{
```

```
    public static void main (String args[])
```

```
{
```

```
    String a = "Vennela";
```

```
    float b = 12.5f;
```

```
    int c = 22;
```

```
    System.out.println("My name is" + a);
```

```
    System.out.println("My cgpa is" + b);
```

```
    System.out.println("My ID is" + c);
```

```
}
```

```
}
```

*. using scanf :-

```
Public class First
{
import java.util.*; → instead of '*' we can also write Scanner
{
Public static void main (String[] args)
{
int a,b,c;

Scanner z = new Scanner (System.in);
System.out.println ("enter a,b values");
a = z.nextInt();
b = z.nextInt();
c = a+b;
System.out.println (c);
}
}
```

*. class :-

A class is a collection of fields (variable) and methods (functions)

Ex: class F303

```
{
int a,b; → Fields
{
Public void klu() → method.
{
}
}
```

Object:

-An object is an instance of a class.

-The properties of a class (fields, methods) can be ^{access} accessed by only the objects which belong to the same class.

-Example for creation of an object.

→ `F303 k = new F303();`

* Create a class projector with the method `light()` and in the method `light` perform the addition of 2 numbers logic. create one more class for the main function and within this class create the object for the class projector and access its properties.

Package FED

import java.util.Scanner;

class Projector

{

public void light() {

{

int a, b, c;

Scanner s = new Scanner(System.in);

System.out.println("enter a, b values");

a = s.nextInt();

b = s.nextInt();

c = a + b;

System.out.println(c);

}

}

```
public class F303
```

```
{
```

```
    public static void main (String[] args)
```

```
{
```

```
        Projector z = new Projector ();
```

```
        z.light();
```

```
    }
```

```
}
```

* Create a class F303. Create the methods fan(), laptop(). Create a class F105 which contains main method and within this class create 2 objects. Use 1st obj for 1st method and 2 obj for 2nd method of class F303.

1. Package FED

2. ~~class F303~~ class F303

3. {

4. public void fan()

5. {

6. System.out.println("Hello Fan");

7. }

8. public void laptop()

9. {

10. System.out.println("Hello laptop");

11. }

12. }

3. public class F105

4. {

5. public static void main (String[] args)

6. {

7. F303 z = new F303();



18. z.fan();

19. F303 S = new F303();

20. S.laptop();

21. y

22. y

* Create a class cse with the method network()

In the method network() write the code for finding biggest of 2 numbers. Create a class ece with the method embedded(). write the code for $v = u + at$ in the method embedded().

Create a class cait with the main() method.

Create the objects for cse and ece class and access their properties.

⇒ ~~class c~~

```
import java.util.Scanner;
```

```
class cse {
```

```
{
```

```
    public void network()
```

```
    {
```

```
        int a, b;
```

```
        Scanner z = new Scanner(System.in);
```

```
        a = z.nextInt(); System.out.println("a, b are");
```

```
        b = z.nextInt();
```

```
        if (a > b)
```

```
        {
```

```
            System.out.println("a is big");
```

```
        }
```

```
        else
```

```
        {
```

```
            System.out.println("b is big");
```

```
        }
```

```
    }
```

class ece

{

public void embedded()

{

int v, u, a, t;

Scanner s = new Scanner(System.in);

System.out.println("u, a, t");

u = s.nextInt();

a = s.nextInt();

t = s.nextInt();

v = u + a * t;

System.out.println(v);

}

}

public class CSIT

{

public static void main(String[] args)

{

CSE k = new CSEC();

k.network();

ECE j = new ECEC();

j.embedded();

}

}

* Static method:

Static methods gets executed before the creation of object. (or) For static methods no need to create an object.

Ex: Accessing static method belonging to

Other class.

```
Package myfirstone;
class cse
{
    public Static void car()
    {
        System.out.println("Hello Car");
    }
}
class
public class First {
    public static void main (System args[])
    {
        cse.car();
    }
}
```

*. Create a class cse with a non-static method projector(). Create one more static method car() within the same class. create a separate class for the main method and there access the properties of the class cse.

```
=> public class First cse
{
    public void projector()
    {
        System.out.println("Hello");
    }
    public static void car()
    {
        System.out.println("Bye");
    }
}
```

public class First

{

public static void main(String args[])

{

cse.car();

cse.^kProjector = new ^{cse}~~Projector~~ ();

k.projector();

}

}

* Accessing static method within the same class

→ public class First

{

public static void main (String args[])

{

car();

}

public ^{static} void car()

{

System.out.println ("car");

}

}

* Static block:

Static block is a block which

get's executed before the main method.

ex:

static

{

}

Example program for static block:

```
Public class First
```

```
{
```

```
    Public Static void main (String[] args)
```

```
{
```

```
    System.out.println ("Hello main");
```

```
}
```

```
    Static
```

```
{
```

```
    System.out.println ("Hello static");
```

```
}
```

```
}
```

Output: Hello static,
Hello main.

→ In the above output we can clearly see that the result of static block is getting executed before the result of main method.

* Modularization: Modularisation can be of 3 types.

1. within the class modularisation.)

2. class level modularisation.

3. package level modularisation

* within the class modularization:

In this kind of modularisation the class will be the single class and within the single class there are multiple methods.

ex:

```
Package FED;  
Public class First  
{  
    Public static void main(String[] args)  
    {  
        First K = new First();  
        K.car();  
        Pen();  
    }  
}
```

Public void car() → non static method of same class

{
 System.out.println("car");
}

static
Public void pen() {
 System.out.println("Pen");
}

static method of same class.
if we keep static no need to create an object.
simply we can write the method "pen()" in main method.

→ object is created for non-static class.

* class level modularisation:

[In class level modularisation within the same package different classes can be created with different file names (each class in each file).
all the classes can be accessed with their respective objects as they belong to same class, package.

Ex:-

* File name : pavan.java

Package library;

Public class pavan

{

Public void marker()

{

System.out.println("Hello pavan");

}

}

* File name : teja.java

Package library;

Public class teja

{

Public void pen();

{

System.out.println("Hello teja");

}

}

* File name : mani.java

Package library;

Public class mani

{

Public static void main (String args[])

{

Pavan k = new Pavan();

k.marker();

teja z = new teja();

z.pen();

}

}

library		
mani	Pavan	teja
main()	pen()	call()

* Package level modularisation;

For package level modularisation classes may belong to some other package still we can access the classes & properties of one package from the class of another package, by using the import statement.

Ex:- File name: Rupesh.java.

```
Package skilling;
```

```
Public class Rupesh
```

```
{
```

```
    Public void pen()
```

```
{
```

```
    System.out.println("hello aaron");
```

```
}
```

```
}
```

File name: mani.java.

```
Package library;
```

```
Package Import skilling.Rupesh; // import skilling.*;
```

```
Public class Mani
```

```
{
```

```
    Public static void main(String[] args)
```

```
{
```

```
    Rupesh z = new Rupesh();
```

```
    z.pen();
```

```
}
```

```
}
```

* In place of Rupesh we can also keep '+'

'+' → means we can access all the classes in that package.

* Fields (or) member variables (or) instance variable :-

Just like methods in a class they can be the fields. The fields in a class can also be considered as a property of a class and those properties can be accessed with the help of objects of that particular class.

* The fields can be both static and non-static and can be accessed same as the methods of a class.

Ex:

```
class Krishna
{
    int id;
    float salary; } → fields.
```

```
public static void showSalary()
{
    System.out.println(salary);
}
```

* The procedure for accessing above properties (fields and methods) of class Krishna we go for object creation.

```
public class jeep
{
    public static void main(String args[])
    {
        Krishna z = new Krishna();
        k.id = 23;
```

```
k.salary = 120000;
```

```
k.showSalary();
```

```
z
```

* Access specifiers :-

Access Spec

provide the security for fields, methods.

they are 4 types of access specifiers they are

1. public (+)
2. private (-)
3. protected (#)
4. default (~)

* public (+) :-

Public Access

applied to a class, method. If the above access specifiers public be used in the same class, through the

* private (-) :-

the private fields can be accessed with in the class. They are not visible outside the class. Also the outside the package.

```
k.Salary = 120000;
```

```
k.showSalary();
```

```
2  
3  
4
```

* Access Specifiers:

Access Specifiers are used to

provide the security for the classes, fields, methods.

they are 4 types of access specifiers are there & they are

1. public (+)
2. private (-)
3. protected (#)
4. default (~)

* public (+):

Public Access specifier can be applied to a class, method(), fields.

If the above are created by using the access specifiers public then they can be used in the same class, from the another class, through the another package.

* private (-):

The private fields and methods can be accessed only in the same class and are not visible outside the class and also outside the package.

default (*)

default keyword can be used for class, methods, fields. If no access specifier is mentioned it means a default access specifier. The things which are created using default access specifier can be accessed within the class, within the package, not to the sub-classes of same package, not to the classes of another package.

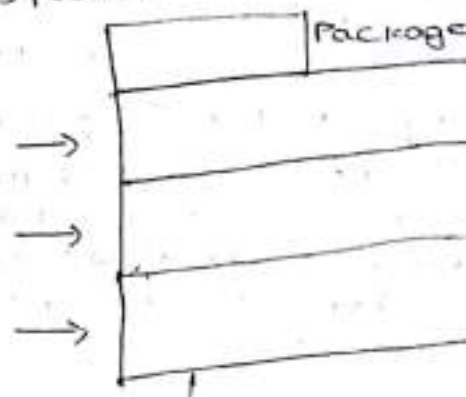
Protected (#)

If a method or a field is declared using protected access specifier then they can be accessed to the sub-classes to of same package but not outside the package.

	with in the class	with in the package	outside the package
Public	✓	✗	✗
Private	✓	✓	✓
Protect	✓	✓ (only if it is child class)	✗
Default	✓	✓	✗

Class diagram

In class diagram, we write class name, field name with a rectangle which is divided into 3 parts.



* consider the following class and draw the class diagram the same.

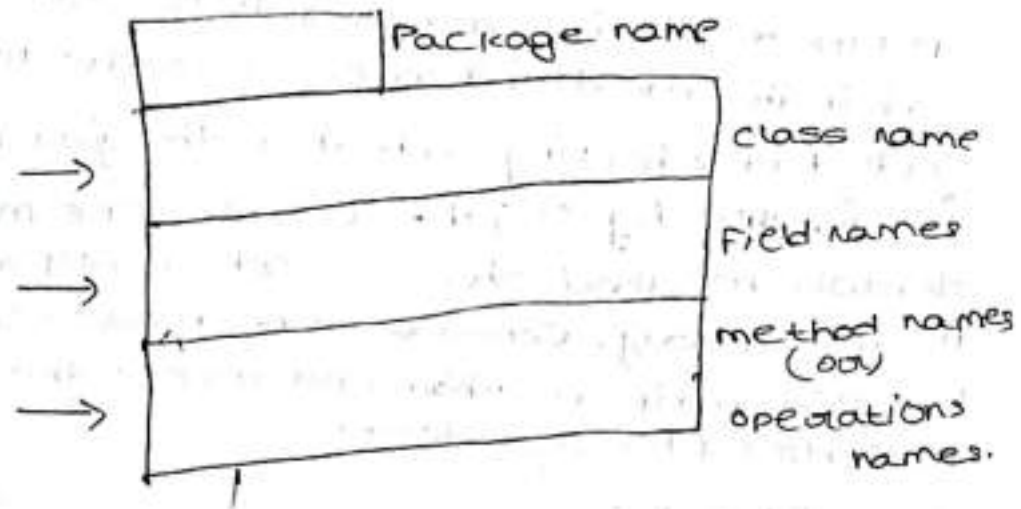
```
Package CSE block;  
class 209  
{  
    public int id;  
    private double salary;  
    public void calc()  
    {  
    }  
    public static void bike()  
    {  
    }  
}
```

underline represents
Indication of static.

CSE block
~ class
+ id : int
- salary
+ calc()
+ bike()

Class diagram :-

In class diagram we represent class name, field names, method names with a rectangle which is internally divided into 3 parts.



*. consider the following example of a class and draw the class diagram for the same.

Package Cse block;

Class 209

{

Public int id;

Private double salary;

Public void car()

{

}

Public static void bike()

{

} }

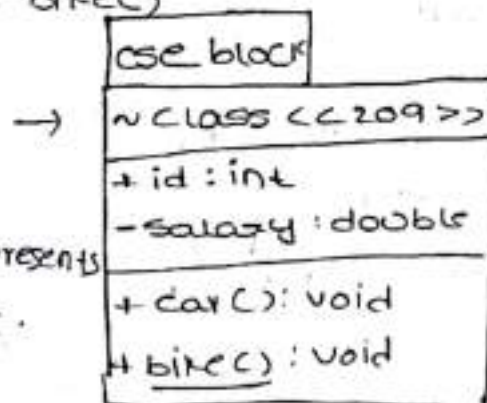
+ - Public

- - Private

- Protected

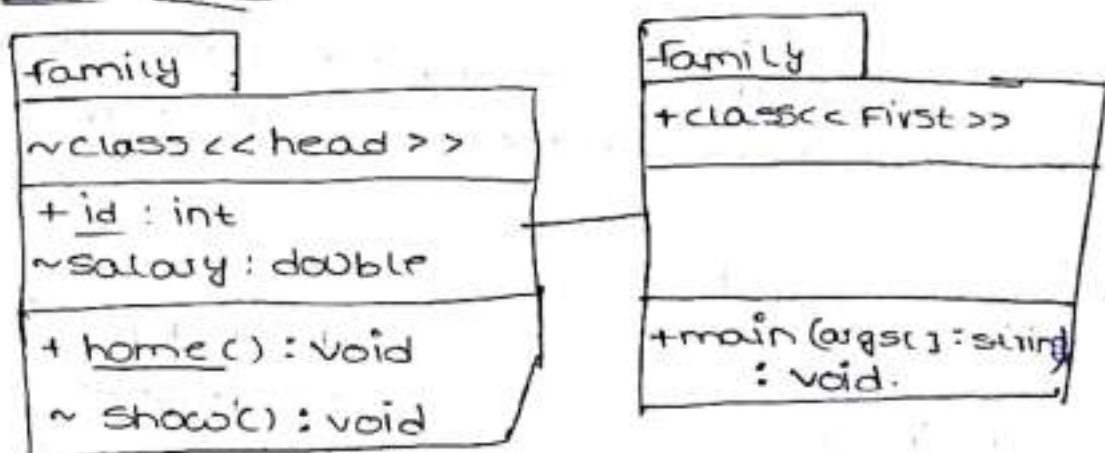
~ - default

() underline represents
Indication of Static.



* Draw the class diagram for the package family. Create a class Head(), with a public static instance variable ID, default field salary of data-type double. Create a method home() with access specified public and also it is static. with in the method home(). write the code for checking whether the given number is divisible by 3 & 7. Create one more default method show(), which displays ID and salary. Create one more class for the main method and access the properties of class Home().

class diagram:



Code:-

```

Package family;
Import java.util.Scanner;
Class head
{
    public static int id;
    double salary;
    public static void home
    {
        Scanner z = new Scanner(System.in);
    }
}
  
```

```
System.out.println("enter n");
```

```
n = z.nextInt();
```

```
if (n % 5 == 0 && n % 7 == 0)
```

```
System.out.println("divisible");
```

```
else
```

```
System.out.println("not");
```

```
}
```

```
show  
public static void homeC(),
```

```
{
```

```
System.out.println(id);
```

```
System.out.println(salary);
```

```
}
```

```
}
```

```
public class First {  
    public static void main (String args[])
```

```
{
```

```
head k = new head();
```

```
head.id = 12;
```

```
k.salary = 12345;
```

```
head.homeC();
```

```
k.showC();
```

```
}
```

```
}
```

Compile time polymorphism:-

Method overloading :-

Methods with same name but with different signature (diff no. of parameters) is said to be method overloading.

Ex:-

```
Package cse;  
class Second  
{  
    Public void vennela()  
    {  
        System.out.println("Hello");  
    }  
    Public void vennela(int a)  
    {  
        System.out.println(a);  
    }  
}  
Public class First  
{  
    Public static void main (String args[])  
    {  
        Second z = new Second ();  
        System.out.println ("Enter a");  
        vennela  
        K.indraAI(34);  
        K.vennela();  
    }  
}
```

- * Create a class Second with overloaded methods `isEven()`, `isEven(int n)`, and check the numbers are even or odd. create one more class for the main method and complete the logic by creating the object.

```

Package CSE;
import java.util.Scanner;
class Second
{
    public void iseven()
    {
        int n;
        Scanner s = new Scanner(System.in);
        System.out.println("enter n");
        n = s.nextInt();
        if(n%2 == 0)
        {
            System.out.println("even");
        }
        else
        {
            System.out.println("odd");
        }
    }

    public void iseven(int n)
    {
        if(n%2 == 0)
        {
            System.out.println("even");
        }
        else
        {
            System.out.println("odd");
        }
    }
}

public class First
{
    public static void main(String[] args) {
        Second k = new Second();
        k.iseven();
    }
}

```

```

int n;
Scanner s = new Scanner (System.in);
n = s.nextInt();
Second z = new second ();
System.out.println ("enter n value in main");
z.isEven(n);
}
}

```

* Array :-

An array is a collection of elements of similar data types.

Example for creation of array :-

```
int a[] = new int[5];
```

* WAIP * create an array with size 5 and read the elements into the array and display those elements.

```

Package cse;
class - second
import java.util.Scanner;
Public class First
{
Public static void main (String args[])
{
int a[] = new int [5];
Scanner s = new Scanner (System.in);
int i;
System.out.println ("enter array elements");
for (i=0; i<a.length; i++)
{
a[i] = s.nextInt();
}
}

```

```

for(i=0; i<a.length; i++)
{
    System.out.println(a[i]);
}
}
}

```

* WAIP to create an array and read the elements into an array and calculate the sum of array elements and display the result.

```

Package cse;
import java.util.Scanner;
Public class Ctod
{
    Public static void main(String[] args)
    {
        int n;
        Scanner z = new Scanner(System.in);
        n = z.nextInt();
        int a[] = new int(n);
        int i, x=0;
        System.out.println("enter array elements");
        for(i=0; i<a.length; i++)
        {
            a[i] = z.nextInt();
        }
        for(i=0; i<a.length; i++)
            x = x + a[i];
        System.out.println(x);
    }
}

```

* Creation of 2d array :-

```
int a[] [] = new int [2] [2];
```

* WAP to create a 2d array read the elements into an array. calculate the sum of elements then display the result.

```
Package Cse;  
import java.util.Scanner;  
Public class Ctod  
{  
    Public Static void main (String [] args)  
    {  
        int a[] [] = new int [2] [2];  
        int i, j, x = 0;  
        Scanner z = new Scanner (System.in);  
        for (i = 0; i < a.length; i++)  
        {  
            for (j = 0; j < a.length; j++) {  
                a[i][j] = z.nextInt();  
            }  
        }  
        for (i = 0; i < a.length; i++)  
        {  
            for (j = 0; j < a.length; j++)  
            {  
                x = x + a[i][j];  
            }  
        }  
        System.out.println (x);  
    }  
}
```

* WAP to create a class abc. If the method $\text{mani}(\text{int } a, \text{int } b)$, which takes the input value from the calling function and return back the result. Create one more class Program with the main method and then complete the accessing procedure for using $\text{mani}()$ method which belongs to the class abc.

```
Package cse;
```

```
class abc
```

```
{
```

```
    Public int mani(int a, int b)
```

```
    {
```

```
        return a+b;
```

```
    }
```

```
}
```

```
Public class program
```

```
{
```

```
    Public static void main (String[] args)
```

```
    {
```

```
        abc s = new abc ();
```

```
        int z;
```

```
        z = s.mani (10,10);
```

```
        System.out.println(z);
```

```
    }
```

```
}
```

```
workbook w-2(1):
```

```
Package ece;
```

```
import java.util.Scanner;
```

```
Public class Armstrong
```

```
{
```

```

public static void isArmstrong ()
{
    int n, r, x=0, z;
    Scanner z = new Scanner (System.in);
    System.out.println ("enter n");
    n = z.nextInt();
    z = n;
    while (n > 0)
    {
        r = n % 10;
        z = z + r * r * r;
        n = n / 10;
    }
    System.out.println (z == z)
    {
        System.out.println ("Armstrong");
    }
    else
    {
        System.out.println ("Not");
    }
}

```

```

public static void display Armstrong ( )
{
    int n, i, r, x=0, z;
    Scanner z = new Scanner (System.in);
    System.out.println ("enter n");
    n = z.nextInt();
    for (i = 0; i <= n; i++)
    {
        x = 0;
    }
}

```

```

z = i;
while (z > 0)
{
    r = z % 10;
    x = x + r * 10 * r;
    z = z / 10;
}
if (x == i)
    System.out.println(i);
}
}
Public static void main (String[] args)
{
    isArmstrong();
    displayArmstrong();
}
}

```

* Develop ATP to create a class ABC with a method show(). which is having array as parameter and display the elements of array. create one more class for the main method where you need to create an array and i/p is to be provided to console. create an object for the class ABC and pass the array to the method show() using parameters.

```

Package ece;
import java.util.Scanner;
Class ABC;
{
    Public void show(int a[])
    {
        int i;
    }
}

```

```

        for(i = 0; i < a.length; i++)
            System.out.println(a[i]);
    }
}

Public class Amotrong
{
    Public static void main (String[] args)
    {
        int a[] = new int[5];
        int i;
        Scanner z = new Scanner (System.in);
        System.out.println("enter elements");
        for(i=0; i < a.length; i++)
        {
            a[i] = z.nextInt();
        }
        Abc k = new Abc();
        k.show (a);
    }
}

```

* Constructors;

Constructors are used to initialize the objects.

⇒ IF the method name is same as the class name then it is a constructor not a method.

⇒ constructor must not have return type.

example :-

```
Package ece;
```

```
class marker
```

```
{
```

```
    Public marker()
```

```
    {
```

```
        System.out.println("Hello");
```

```
    }
```

```
}
```

```
class public class Armstrong
```

```
{
```

```
    Public static void main (String[] args)
```

```
    {
```

```
        marker k = new marker();
```

```
    }
```

```
}
```

* Types of Constructors:- there are 3 types
of constructors are there.

1. Default constructor.
2. Zero parameter constructor.
3. parameterized constructor.

* Default constructor:-

Default constructors are the constructors which are used for object creation.

* Zero parameterized constructor:-

Constructors with zero parameters is called zero parameter constructor.

ex:- Package ece;

```
class marker
```

```
{
```

```
    Public marker()
```

```
    {
```

```
System.out.println("Hello");
```

```
}
```

```
}
```

```
Public class Armstrong
```

```
{
```

```
Public static void main(String[] args)
```

```
{
```

```
    marker k = new marker ();
```

```
}
```

```
}
```

* Parameterized Constructor:-

Constructor with atleast one parameter (1 or more) is said to be parameterized constructor.

Ex:- package ecc;

class marker

```
{
```

```
    Public marker(a)
```

```
{
```

```
    System.out.println(a);
```

```
}
```

```
}
```

```
Public class Armstrong
```

```
{
```

```
Public static void main(String[] args)
```

```
{
```

```
    marker k = new marker (10);
```

```
}
```

```
}
```

* Constructor Overloading:-

Constructors with same name but with different signature is said to be Constructor Overloading.

Ex:-

```
Package ece;
class marker
{
    public marker()
    {
        System.out.println("A zero param const");
    }
    public marker(int a)
    {
        System.out.println("a");
    }
    public marker(int a, int b)
    {
        System.out.println(a+b);
    }
}

Public class Armstrong
{
    public static void main (String[] args)
    {
        marker k = new marker();
        marker z = new marker(10);
        marker s = new marker(10, 10);
    }
}
```

Constructor chaining using this() method:-

It is a process of calling one constructor from another constructor. this can be done by using a this() method. constructor chaining is possible only in the case of constructor overloading.

Ex:-

```
Package ece;  
class marker  
{  
    public marker()  
    {  
        this(10);  
        System.out.println("zero param");  
    }  
}
```

```
    public marker(int a)  
    {  
        this(10, 10);  
        System.out.println(a);  
    }  
}
```

```
    public marker(int a, int b)  
    {  
        this  
        System.out.println(a+b);  
    }  
}
```

```
}
```

```
}
```

```
public class Armstrong
```

```
{
```

```
    public static void main(String[] args)
```

```
{
```

```
        marker k = new marker();
```

```
    }
```

```
}
```

OP:-

20

100

zero param

Setter and getter methods;

Setter - Mutators

Getter - Accessors

⇒ In order to access the private attributes of a particular class there is no way from other classes to access them

but,

we have setters and getters which can help to ^{pass} ~~pass~~ the values through parameters (through) and get the values.

⇒ Setter methods are used for setting values to the private attributes of a class through its parameters.

⇒ Getter method can help to return the obtained value in the private attributes of a class.

Example:-

```
class marker
{
    private int id;
    public void setId (int id) - 20
    {
        this.id = id; (this keyword is used for accessing
                     private attributes of a class).
    }
    public int getId()
    {
        return id;
    }
}
```

-this keyword:-

-this keyword is used to access the Private attributes (data members or) member variables (or) fields of a class.

Ex:-

-this.id;

-this.salary; this.name;

- * CAJ class marker with a private attribute id. provide the value to that private attribute id using the setter method and access that using that getter method. Create one more class Vennela with the main method. create an object for the class marker and complete the requirement of providing the value and displaying the given value.

```
Package ece;  
class marker  
{  
    Private int id;  
    Public void setid (int id)  
    {  
        this.id = id;  
    }  
    Public int getid ()  
    {  
        this.id = id; return id;  
    }  
}  
Public class Vennela.  
{  
    Public static void main (String[] args)
```

```

{
    makeK k = new makeK();
    k.setId(10);
    System.out.println(k.getId());
}
}

```

* Develop a class setget with the private attribute id, name, salary. create setter and getter methods for accessing the private attributes and getting back the provided inputs. create one more class solution with the main method. create the object here and complete the requirements of a program.

```

package ece;

class Setget
{
    private int id;
    private String name;
    private double salary;

    public void setId (int id)
    {
        this.id = id;
    }

    public void setName (String name)
    {
        this.name = name;
    }

    public void setSalary (String double salary)
    {
        this.salary = salary;
    }

    public int getId()
    {
        return id;
    }
}

```

```

public String getName ( )
{
    return name;
}

public double getSalary ( )
{
    return salary;
}

}

class public class solution
{
    public static void main (String[] args)
    {
        int
        Setgel k = new Setget();
        k.setid (25);
        k.setName ("vennela");
        k.setSalary (12500);

        System.out.println (k.getId);
        System.out.println (k.getName);
        System.out.println (k.getSalary);
    }
}

```

to string () method :-

to string () method returns the value to the object of that particular class whenever we prints the object the values which are returned by the toString () method get's displayed.

1. toString() method can be overridden.

2. Create a class getsetgo, with private instance variables id, name. Create setter and getter methods for those instance variables use toString() method to return the results to the object. Create one more class for the main method. Complete the access formalities of for the class getsetgo and display the result.

```
Package ecc;  
class getsetgo  
{  
    Private int id;  
    Private String name;  
    Public void setId (int id)  
    {  
        this.id = id;  
    }  
    Public int getId ()  
    {  
        return id;  
    }  
    Public void setName (String name)  
    {  
        this.name = name;  
    }  
    Public String getName()  
    {  
        return name;  
    }  
    Public String toString()  
    {  
        return "id = " + id + " name = " + name;  
    }  
}
```

{

Public Static void main (String[] args)

{

getsetga k = new getsetgo();

k.setid(12);

k.setName("KLU");

System.out.println(k);

}

}

* Inheritance:- deriving a child class from the parent class is known as Inheritance

(or)

Parent

Deriving the properties of ^{Parent} child class from

to the child class. using extends ^{keyword} we can apply

Alternate names for parent class:-

1. Parent class
2. Base class
3. Super class

Alternate names for child class

1. Sub class
2. child class
3. derived class.

* Types of inheritance:-

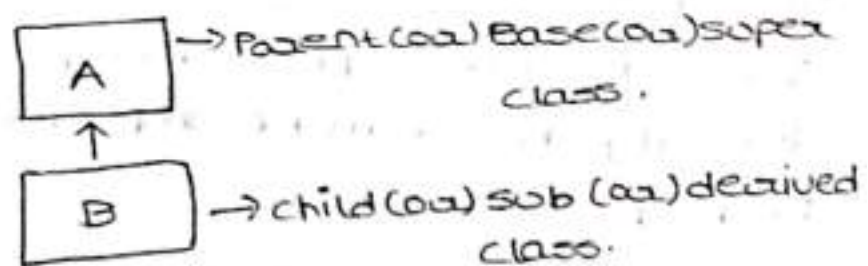
There are 4 types of inheritance

these they are

1. Single inheritance.
2. Multilevel inheritance.
3. Hierarchical inheritance.
4. Multiple inheritance.

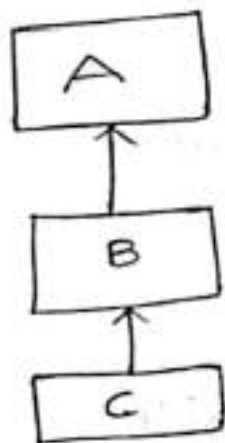
* Single inheritance:-

Deriving a single child class from single parent class is known as single inheritance.



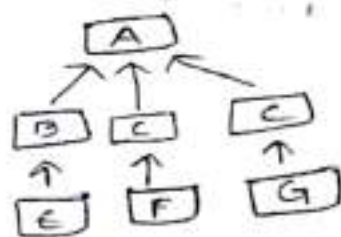
* Multi level inheritance:-

Deriving a child class from the Parent class which is already derived from its parent class is known as multi level inheritance.



* Hierarchical inheritance:-

Deriving more than 1 child class from single parent class is called Hierarchical inheritance.



4. Multiple Inheritance:-

Deriving a single child class ^{from} for

more than one parent class is known as multiple inheritance

=> Java doesn't support multiple inheritance directly, but can be implemented with the help of interfaces.

Example: (Single Inheritance).

```
*. Package myfirstone;  
class Parent  
{  
    Public void car()  
    {  
        System.out.println("hello");  
    }  
}  
class child extends Parent  
{  
    Public void bike()  
    {  
        System.out.println("hai");  
    }  
}
```

```
Public class Inheritance.
```

```
{  
    Public static void main (String[] args)  
    {  
        Child k = new child();  
        k.bike();  
        k.car();  
    }  
}
```

* Create a class cuboid with a method volume (int l, int b, int h) which takes the input values l, b, h through parameters, and display volume of a cuboid. Create a child (or) subclass rectangle with a method Area (int l, int b) which receives l, b values through parameters and display the area of rectangle. Create one more main method class where create an object for child class rectangle and access the properties of both parent and child class. Pass the I/P values from the main method by taking I/P values from console.

```

=> Package myfirstone;
    class cuboid → import java.util.Scanner;
    {
        public void volume (int l, int b, int h)
        {
            System.out.println (l*b*h);
        }
    }
    class rectangle extends cuboid
    {
        public void Area (int l, int b)
        {
            System.out.println (l*b);
        }
    }
    public class Inheritance
    {
        public static void main (String[] args)
        {
            int l, b, h;
            Scanner s = new Scanner (System.in);
            l = s.nextInt();

```

```
b = s.next Int();
```

```
h = s.next Int();
```

```
Rectangle z = new Rectangle();
```

```
z.area(l, b);
```

```
z.Cuboid(l, b, h);
```

```
}
```

```
}
```

* Multilevel Inheritance:

WATP for multilevel inheritance where create a Parent (or) Super class ^{Cuboid} with a method volume (int l, int b, int h) display volume of Cuboid. create a child class rectangle in that method area (int l, int b) which displays area of rectangle. create one more child class square which extends class rectangle. Create a method areasquare (int s) which displays the area of square. Create one more class for the main method in which create an object for the class ~~square~~ square and access the properties of all the available classes.

```
=> Package myfirstone;  
import java.util. scanner;  
Class Cuboid  
{  
    Public void volume(int l, int b, int h)  
    {  
        System.out.println (l*b*h);  
    }  
}
```

```
class rectangle extends cuboid
```

```
{  
    public void area(int l, int b)  
{  
    System.out.println(l*b);  
}
```

```
}  
}
```

```
class Square extends rectangle
```

```
{  
    public void areaSquare(int s)  
{  
    System.out.println(s*s);  
}
```

```
}  
}
```

```
public class Inheritance
```

```
{  
    public static void main(String[] args)  
{  
        int l, b, h, s;  
        Scanner z = new Scanner(System.in)  
        l = z.nextInt();  
        b = z.nextInt();  
        h = z.nextInt();  
        s = z.nextInt();  
  
        Square m = new Square();  
        m.area(l, b);  
        m.areaSquare(s);  
        m.volume(l, b, h);  
    }  
}
```

```
Square m = new Square();
```

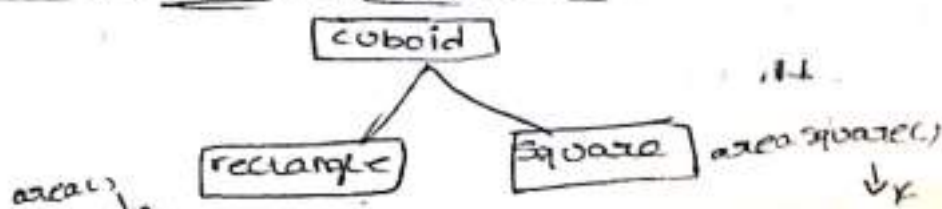
```
m.area(l, b);
```

```
m.areaSquare(s);
```

```
m.volume(l, b, h);
```

```
}  
}
```

* Hierarchical Inheritance :



```

Package myfirstone ;
import java.util.Scanner;
class cuboid
{
    public void volume (int l, int b, int h)
    {
        System.out.println (l*b*h);
    }
}
class rectangle extends cuboid
{
    public void area (int l, int b)
    {
        System.out.println(l*b);
    }
}
class Square extends cuboid
{
    public void areaSquare(int s)
    {
        System.out.println (s*s);
    }
}
public class Inheritance
{
    public static void main (String[] args)
    {
        int l, b, h, s;
        Scanner m = new Scanner (System.in);
        l = m.nextInt();
        b = m.nextInt();
        h = m.nextInt();
        s = m.nextInt();
    }
}

```

```
Square k = new Square();
```

```
k.areaSquare(s);
```

```
k.volume(l, b, h);
```

```
rectangle z = new rectangle();
```

```
z.area(l, b);
```

```
z.volume(l, b, h);
```

```
}
```

```
}
```

* Run time polymorphism:-

Polymorphism which can be done or applied during run-time of a program. Method overriding is an example for run time polymorphism.

* Method overriding:-

Method with same name ~~but~~ with ^{same} ~~different~~ signature one in parent class and another in child class is known as method overriding.

example program for method overriding:-

```
Package cse;
```

```
Class cuboid
```

```
{
```

```
Public void calc()
```

```
{
```

```
System.out.println("hello");
```

```
}
```

```
}
```

```
Class rectangle extends cuboid
```

```
{
```

```
Public void calc()
```

```
{
```

```

}
System.out.println("hai");
}
}

```

Public class Inheritance

```

{
    public static void main(String[] args)
    {
        Rectangle k = new Rectangle();
        k.draw();
        Cuboid z = new Cuboid();
        z.draw();
    }
}

```

=> the above example we are using method name `draw()` for implementing for method overriding, here ^{by} the implementing method overriding the no. of parameters must be same

* final keyword:-

final keyword can be applied to

- i. To the class
- ii. the method
- iii. the variables.

final keyword for the class:-

If we make a particular class as final then that class cannot be inherited or ~~the class~~ we cannot create a child class for the final class.

ex:-

```

Package cse;
final class Cuboid

```

```

{
    public void calc()
    {
        System.out.println("hello");
    }
}

```

```

class rectangle extends cuboid
{
    }
}

```

error: final class cannot be extended.

* final keyword for the method:

If we apply the final keyword for a particular method the method overriding is not possible.

Ex:-

```

package cse;
class cuboid
{
    public final void calc()
    {
        System.out.println("hello");
    }
}
class rectangle extends cuboid
{
    public void calc() → error; trying to override the method.
    {
        System.out.println("Hi");
    }
}

```

* Final keyword for the variable:

If we apply final keyword for a variable then the value of a variable remains constant and cannot be updated.

Ex:- final int a=12;

a=43; → this line gives error as we are trying to update the value of a - final variable.

* Super keyword:

Super keyword is used to access the properties of parent class from child class

Ex:-

```
Package Cse;
class cuboid {
    int id;
    public void train()
    {
        System.out.println("Hello train Id");
    }
}

class rectangle extends cuboid
{
    public void call()
    {
        super.id = 43; → Parent class variable.
        super.train(); → Parent class method()
        System.out.println("Hello call");
    }
}

public class Inheritance
{
    public static void main(String args[])
    {
        rectangle r = new rectangle();
    }
}
```

```
k.calc();
```

```
}  
}
```

Super method :-

super() method is used to access Parent class constructor from child class constructor.

Ex:-

```
Package cse;
```

```
class cuboid
```

```
{
```

```
    public cuboid()
```

```
{
```

```
        System.out.println("hello");
```

```
}
```

```
}
```

```
class rectangle extends cuboid
```

```
{
```

```
    public rectangle()
```

```
{
```

```
        super();  $\rightarrow$  using this method we are accessing the
```

```
        System.out.println("hai");
```

Parent class.

```
}
```

```
}
```

```
public class Inheritance {
```

```
{
```

```
    public static void main(String args[])
```

```
{
```

```
        rectangle k = new rectangle();
```

```
}
```

```
}
```

Dynamic method dispatch (DMD):

In case of method overriding

where all the parent and child classes contains the same method name, and if we have to decide which particular overridden method is to be executed, dynamically we can decide that. Using Dynamic method dispatch (DMD), we can full fill the above mentioned requirements.

Example:-

```
Package cse;
class cuboid
{
    public void carc()
    {
        System.out.println("hello");
    }
}
class rectangle extends cuboid
{
    public void carc()
    {
        System.out.println("hai");
    }
}
public class Inheritance
{
    public static void main (String args[])
    {
        cuboid p;
        rectangle z = new rectangle();
        cuboid s = new cuboid();
    }
}
```

```

P = 2;
P.setColor();
P = 5;
P.setColor();
}
}

```

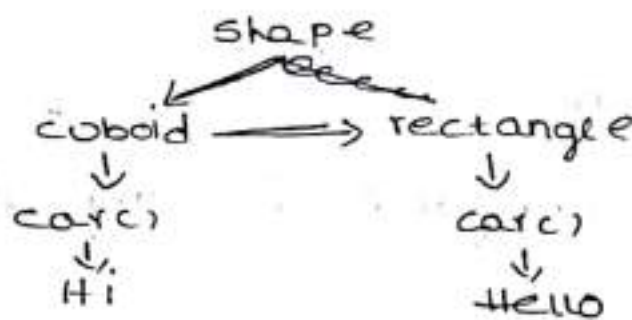
* Other way of applying DMA:

```

public static void main(String args[])
{
    Shape k = new ShapeCoboid();
    k.setColor();

    Shape p = new rectangle();
    k.setColor();
}

```



OP: Hi
Hello.

* Command line Arguments (CLA):

CLA are the inputs to be provided through the parameter of main method.

ex:-

* WAIP to calculate sum of elements in an array and display the sum. input is to be provide from through CLA

```

Package ece;
public class one
{
    public static void main (String args[])

```

```

{
    int i, x = 0;
    for (i = 0; i < args.length; i++)
    {
        x = x + Integer.parseInt (args[i]);
    }
    // Step to convert string to int.
    System.out.println (x);
}
}

```

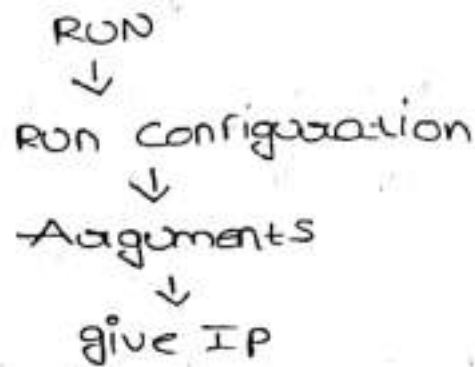
- * WAIP to calculate sum of even elements in an array by providing input through CLA

```

Package ece;
Public class one
{
    Public static void main (String args[])
    {
        int i, x = 0, z;
        for (i = 0; i < args.length; i++)
        {
            z = Integer.parseInt (args[i]);
            if (z % 2 == 0)
            {
                x + x = x + z;
            }
        }
        System.out.println (x);
    }
}

```

How to give ip in CLA:



* Wrapper Class:-

In java there are certain classes created for the respective datatypes (Primitive data types) and these classes are said to be wrapper classes.

→ If we want to create an object for a particular data type we cannot do that by using primitive datatypes like int, char, float etc ... instead we can do that by using wrapper classes.

Primitive data-type	Wrapper class
int	Integer
char	Character
float	Float
double	Double
boolean	Boolean
byte	Byte
long	Long
short	Short

* Auto boxing & Unboxing :

The process of conversion of Primitive datatype into the wrapper class is known as Auto boxing.

⇒ The process of conversion of wrapper class back into primitive datatype is known as unboxing.

* WAIP to read integer number into an integer variable and then convert into (s) String into and Integer object by using :
- Auto boxing.

```
Package cse;
```

```
Public class one
```

```
{
```

```
Public static void main (String args[])
```

```
{
```

```
int a=25
```

```
Integer z;
```

```
z = Integer.valueOf(a); → Auto boxing step
```

```
System.out.println(z);
```

```
}
```

```
}
```

* WAIP for auto and unboxing.

```
Package ece;
```

```
Public class one
```

```
{
```

```
Public static void main (String args[])
```



```
{  
    int a=23, p;
```

```
    Integer z;
```

```
    z = Integer.valueOf(a); — Auto boxing  
    System.out.println(z);
```

```
    p = z.intValue(); — unboxing  
    System.out.println(p);
```

```
}
```

```
}
```

*. WAP to find the area of circle by getting the input from command line argument.

```
Package ecc;
```

```
Public class one
```

```
{
```

```
    Public static void main (String args[])
```

```
{
```

```
    float ar;
```

```
    r = Float.parseFloat(args[0]);
```

```
    a = (float) (3.14 * r * r);
```

```
    System.out.println(a);
```

```
}
```

```
}
```

*. Array of Objects:

In order to provide the

details of multiple employees or multiple students or details of multiple persons we need to create multiple objects.

if the requirement is multiple of 100 then it is impossible to create those many no. of objects.

Due to this issue we go for creation of array of objects.

Example:-

for creation of object.

```
Abc a[] = new Abc[5];  
int i;      ↳ class name.
```

```
for(i=0; i<a.length; i++)  
{  
    a[i] = new Abc(); ↳ step for assignment of  
                        constructor.  
}
```

* create a class employee with private attribute id, use setter (), getter (), toString () methods. Create one more class One with the main method. Create an array of objects and read the details of 5 employees and display them
Package ecc;

```
import java.util.Scanner;
```

```
class employee
```

```
{
```

```
    private int id; ↳ scanner import java.util.  
    public int getId() Scanner;
```

```
{
```

```
    return id;
```

```
}
```

```
    public void setId(int id)
```

```
{
```

```
        this.id = id;
```

```
}
```

```
    public String toString()
```

```
{
```

```
        return "id = " + id;
```

```
}
```

public class one

{

public static void main(String args[])

{

employee a[] = new employee[5];

Scanner s = new Scanner(System.in);

int i, id;

for(i=0; i<a.length; i++)

{

a[i] = new employee();

}

for(i=0; i<a.length; i++)

{

System.out.println("Enter id");

id = s.nextInt();

a[i] = setId(id);

}

for(i=0; i<a.length; i++)

System.out.println(a[i]);

}

}

- * Create a class student with attributes id, name, marks which are private. Create setter, getter, toString method for all attributes. Create an array of objects in another class and set the details of 5 students and display those details.

Package ecc;

import java.util.Scanner;

class student

{

private int id;

private int String name;

private double marks;

```
Public void setId(int id)
```

```
{
```

```
-this.id = id;
```

```
}
```

```
Public void setName(String name)
```

```
{
```

```
-this.name = name;
```

```
}
```

```
Public void setMarks(double marks)
```

```
{
```

```
-this.marks = marks;
```

```
}
```

```
Public int getId()
```

```
{
```

```
return id;
```

```
}
```

```
Public String getName()
```

```
{
```

```
return name;
```

```
}
```

```
Public double getMarks()
```

```
{
```

```
return marks;
```

```
}
```

```
Public String toString()
```

```
{
```

```
return "id = " + id + " name = " + name + " marks: " + marks;
```

```
}
```

```
}
```

```
Public class one
```

```
{
```

```
Public static void main (String args[])
```

2

```
Student a[] = new Student[5];  
Scanner s = new Scanner(System.in);  
int i, id;  
String name;  
double marks;  
for (i = 0; i < a.length; i++)  
{  
    a[i] = new Student();  
}  
for (i = 0; i < a.length; i++)  
{  
    System.out.println("Enter id, name, marks");  
    id = s.nextInt();  
    name = s.next();  
    marks = s.nextDouble();  
    a[i].setId(id);  
    a[i].setName(name);  
    a[i].setMarks(marks);  
}  
for (i = 0; i < a.length; i++)  
    System.out.println(a[i]);  
}
```